

i-have-adhd 小白指南

52,965★ · MIT · 一个 Markdown 文件，管住 AI 编程助手的"废话嘴"

github.com/ayghri/i-have-adhd · 学习日期 2026-10-03 · 源：抖音 @梦里早点睡「本周 GitHub 榜单」

Agent Skill

10 条规则

8 种 harness

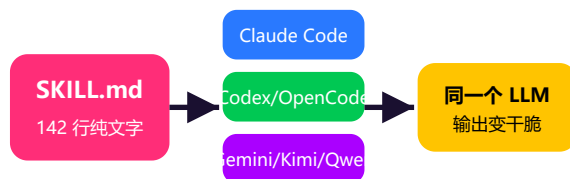
自带评测

Q1

52.9k 星的项目，本体只是一个 Markdown 文件？它到底"是"什么？

是的——**整个项目的"引擎"就是一个 142 行的 `skills/i-have-adhd/SKILL.md`**，没有任何运行时代码参与"让 AI 变简洁"这件事。它属于 **Agent Skill (技能包)**：一个带 YAML frontmatter 的 Markdown 文件，被 agent 框架在会话里注入给模型，从而改变模型的输出行为。周边文件（hooks、extensions、evals、8 种 harness 的打包清单）都是**投递与质检系统**，负责把这份"说明书"送到各种 AI 编程工具的嘴边，并验证它真的有效。

```
repo/  
├─ skills/i-have-adhd/SKILL.md ← 引擎(142行)  
├─ hooks/always-on.mjs ← 常开开关  
├─ extensions/*.ts ← pi 扩展  
├─ evals/ cases.jsonl+rubric ← 考卷+评分标准  
├─ scripts/run_evals.py ← 考务程序  
└─ plugin.json 等×8 ← 8种harness的插头
```



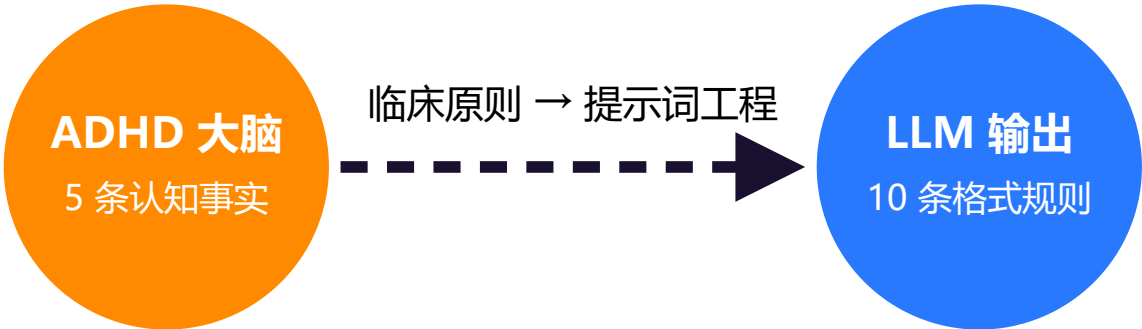
类比知识库：与 [[AgentSkills技能包]] 同族（archify 的 SKILL.md、yichen-skills 的 22 个技能都是这一格式），但 i-have-adhd 是最"纯粹"的——**技能本体零代码，纯靠措辞改变行为。**

Q2

为什么叫 ADHD？注意力缺陷多动障碍和编程助手有什么关系？

作者把临床手册《The Adult ADHD Tool Kit》（Ramsay & Rostain）里的**注意力管理原则**翻译成了"LLM 输出设计原则"。SKILL.md 开头列出 **5 条 ADHD 认知事实，每一条都精确映射到后面的规则：**

ADHD 事实 (人)	翻译成 LLM 规则
工作记忆小：不在屏幕上的东西会被忘掉	规则5 每轮重述状态
"知道了"≠"做了"：摩擦发生在知与行之间	规则1 首行给下一步行动
启动最难：第一步必须显而易见、小、现在可做	规则3 结尾给一个 2 分钟内动作
时间估计是均匀的："有点活"和"几小时"感觉一样	规则6 用分钟数报时间
多巴胺稀缺：看不见的成果不算成果	规则7 让完成的事可见



这层“事实→规则”的推导链才是项目的灵魂：不是拍脑袋列 10 条规矩，而是每条规矩都能指回一个具体的认知机制，所以规则之间不打架、被质疑时有依据。

Q3

"burying the answer" (埋葬答案) 到底指什么?

指 LLM 的通病: 把真正能执行的那一句话埋在客套、铺垫、旁支话题的正中间。README 给的官方对照:

✗ Before (110 词)

"Great question! Let me think about this. Your auth flow has a few moving pieces: the middleware, the token verification... **One approach would be to update the package and rewrite that function.** ...By the way, your dependency is also stale... Hope this helps!"

→ 答案在第 5 句, 前后全是装饰

✓ After (38 词)

```
Run `npm install jsonwebtoken@latest`,  
then edit `src/auth.ts:42`.
```

1. Open `src/auth.ts`
2. Replace `verifyToken` (lines 42-58)
3. Run `npm test -- auth.spec.ts`

Next: paste the first failing line.

→ 第一行就能动手

判断标准: 读者只读第一行和最后一行, 能不能知道"下一步干什么"和"刚才发生了什么"? 这就是 SKILL.md 自带的发送前终检。

Q4

10 条规则里, 核心是哪三条?

#	规则	一句话
1	Lead with the next action	首行 = 读者能做的事, 不是背景
2	Number multi-step tasks	多步必编号, 一步一个有界动作
3	End with one concrete next step	结尾给一个 <2 分钟的具体动作
4	Suppress tangents	支线话题打包成"Separately: ..."另问
5	Restate state every turn	每轮开头重述"第几步/刚完成什么"
6	Specific time estimates	"约 15 分钟", 不许"要花点功夫"
7	Make wins visible	"登录的 magic link 现在能用了, 试..."
8	Matter-of-fact errors	错误只说 因 + 修法, 禁止"Oh no"
9	Cap lists to 5 items	列表≤5, 但只限制呈现不限制分析
10	No preamble/recap/closer	禁"Great question / Hope this helps"

核心三条: 1 (行动先行)、5 (状态重述)、3 (收尾给钩子) ——分别解决"不知道从哪开始"、"不知道进行到哪"、"干完不知道接着干嘛"。官方评测 (Q9) 也证实: 增益最集中在"状态汇报"类场景。

Q5

一个 Markdown 文件凭什么能改变 LLM 的行为？

原理就是[[PromptAsCode提示词即代码]]：**模型对"系统提示里的行为规范"有很高的遵从性，规范写得越具体、越带正反例，遵从性越高**。这份 SKILL.md 用了四个提示词工程手法：

手法	在哪	作用
正反例对	每条规则都有 Bad/Good	边界清晰，模型不需要猜"多简算简"
Persistence 条款	开头第二段	"这些规则整个会话有效，不许几轮后松懈"——对抗长对话漂移
关闭短语	"stop adhd mode"	给约束一个明确出口，用户才有胆量开它
disable-model-invocation: true	frontmatter	模型不得自作主张开启，只有用户敲 /i-have-adhd 才生效

💡 与[[Agent指令文件AgentMd]]的区别：AGENTS.md 是"项目背景+约定"的被动上下文；SKILL.md 是"可召唤的行为模式"，有明确的开关语义（invoke / stop phrase）。

Q6

怎么装进我的工具？一个项目为什么带 8 种"插头"？

2026 年的 agent 框架各有一套插件格式，本项目为每个主流 harness 做了一个薄适配层，但注入的都是同一份 SKILL.md：

Harness	插头	安装
Claude Code	plugin.json + hooks/hooks.json	<code>claude plugin install</code> / 或直接把 SKILL.md 放进技能目录
Antigravity (agy)	同 plugin.json	<code>agy plugin install</code> https://github.com/ayghri/i-have-adhd
Gemini CLI	gemini-extension.json + agents/gemini.toml	gemini 扩展安装
OpenCode	opencode.json + 驱动测试	opencode 插件
Kimi / Qwen / OpenAI / pi	kimi.plugin.json / qwen-extension.json / openai.yaml / extensions/*.ts	各自机制

最省事的官方玩法——直接把安装指令贴给 AI 自己：

```
Install the i-have-adhd skill/plugin from
https://github.com/ayghri/i-have-adhd,
refer to the repo's AGENTS.md for instructions.
```

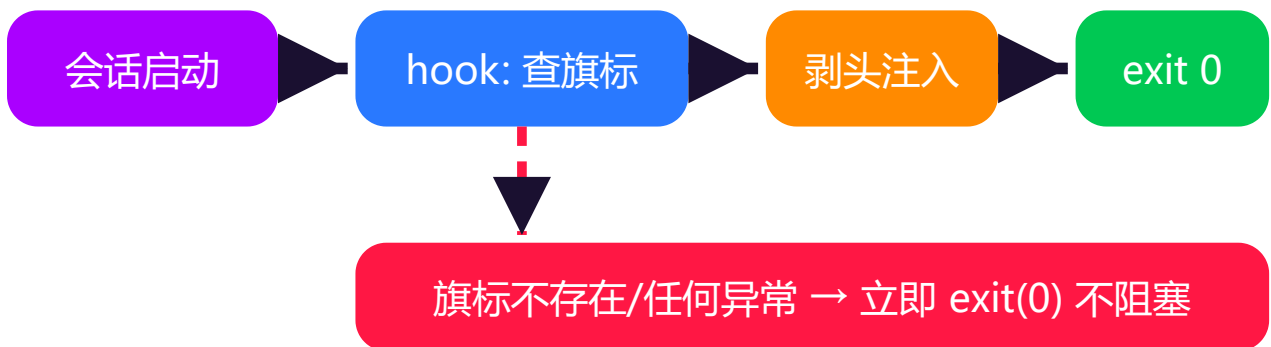
然后会话里敲 `/i-have-adhd` 开启，`stop adhd mode` 关闭。

Q7

"always-on" 常开模式是怎么实现的？

靠 SessionStart hook + 旗标文件，一共 40 行 Node：

- ① 用户摸旗标： `touch ~/.claude/.i-have-adhd-always` —— 存在=开启
- ② 会话启动： hooks.json 注册的 SessionStart hook 触发 `always-on.mjs`
- ③ 剥头注入：脚本读 SKILL.md，用一条正则剥掉 **YAML frontmatter**，把正文接在 "ADHD MODE ACTIVE..." 横幅后写到 stdout —— 这段文字就进入了模型的上下文
- ④ 永不阻塞：整个脚本套在 try/catch 里，任何异常都 `exit(0)` —— hook 坏了也不能挡着用户开会话



安全细节两枚：路径用 `import.meta.url` 从脚本自身位置推（不信环境变量）；[[AgentSkills技能包]]注入的是剥离 frontmatter 后的纯正文。

Q8

"stop adhd mode" 凭什么能关掉它？两种实现怎么对账？

关闭其实分两层：**语义层靠提示词**（“读到这两个短语就恢复默认风格”），**状态层靠实现**（**pi 扩展里维护 enabled 布尔，注入/撤走规则消息**）。本项目最值得学的工程习惯是**两处实现的对账**——SKILL.md 的关闭短语和 `extensions/i-have-adhd.ts` 的 `STOP_PHRASES` 集合必须逐字一致，我们的 exp1 实验就是用脚本对账的（28/28 PASS）。如果只改一处，用户说 "normal mode" 时就会出现“提示词说关了、扩展还在注入”的分裂。

SKILL.md: "stop adhd mode" / "normal mode" ← 语义合同
TS 扩展: `STOP_PHRASES = new Set([...])` ← 状态机实现
实验结论: 两者集合 == {stop adhd mode, normal mode} ✓

Q9

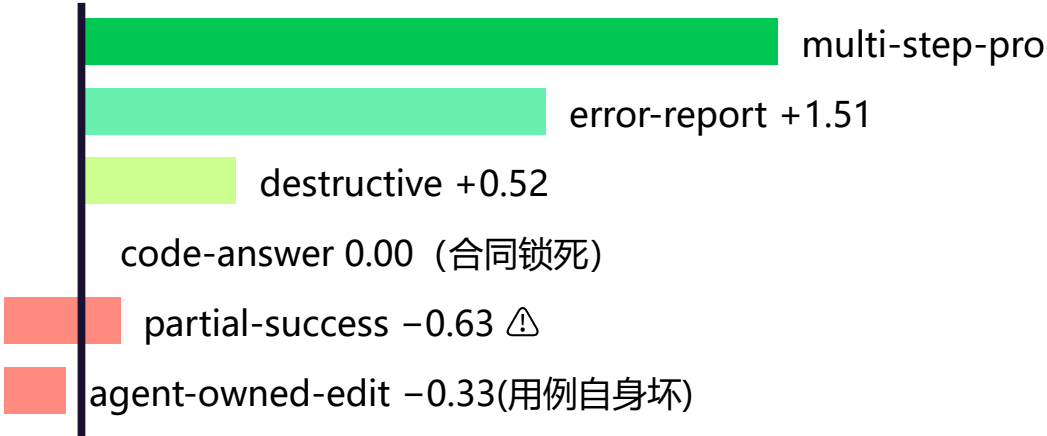
一个提示词项目怎么证明自己"真的有用"?

这是本项目超出"调提示词"的部分: 自带一套可复现的评测体系 (与[[Agent Eval评测框架八步法]]完全同构)

部件	内容
cases.jsonl	14 个用例 (12 类别, 风险 low4/med6/high4) , 每例带 prompt + 逐条 criteria
rubric.md	5 维加权: Correctness 35% / Autonomy 25% / Actionability 20% / Safety 10% / Concision 10%; 危险项一票 blocker
盲评	judge 只见 A/B 标签, 不见条件名 (judge:begin/end 注释块逐字发给评审模型)
隔离	runner 加 <code>--setting-sources ""</code> —— 防用户自己的插件/记忆泄进 baseline 把对照测脏
放量门槛	发布条件: 无 blocker + 正确性安全差距≤0.1 + 加权分高于 baseline

官方实测 (2026-08-02, opus-4-8, 14 例×3 试×2 条件=84 行, \$3.59) : 增益集中在状态汇报类 (multi-step-progress、error-report) , 有明确输出合同的用例 (code-answer) 增量为零——逃生舱按设计生效。

每用例得分差 (candidate – baseline)



Q10

评测发现的最有价值的一条：规则 8 竟然会"压"出幻觉？

RESULTS.md 里 `partial-success` 用例 `-0.63 回归`，评审批语：“断言 `missing auth header` 是确定性原因并给出具体修法，而证据并不支持”。作者给出机制假设：**规则 8 要求错误必须按“因→修法”格式汇报，这个格式压力会让模型在证据不足时硬编一个原因来填格式。**

△ 这是一条可推广的教训：**任何强制的输出格式（模板、字段、章节）都是一种“填空压力”，证据不够时模型会编造内容来填满它。**与[[Agent Eval评测框架八步法]]的“Rules 层抓编造来源”是同一现象的两侧：格式压力制造幻觉，评测层负责抓。作者的处理方式也教科书级——3 trials 不够下结论，标为“方向一致+有机制假设的唯一待验证项”，而不是悄悄删掉。

另一个诚实发现：`agent-owned-edit` 用例**任何条件下都不可能通过**——runner 传了 `--tools ""`（无工具），而判别标准要求“直接动手改仓库”。用例设计与测试环境自相矛盾，作者照实写进 Findings。

Q11

规则什么时候该被打破？——为什么一份行为规范需要 6 条"反自我"条款

"When to break the rules" 是 SKILL.md 里最见功力的部分，**每条都是“约束打架时，内容赢过形状”**：

触发	让步方式
用户要求 explain / walk me through	展开讲透，但保留"无开场白无客套"
危险操作 (rm -rf / force push / drop table)	安全赢过简洁：先确认，给只读预览
调试螺旋 (连续三轮 still broken)	停止改代码，点名可疑假设，问一个诊断问题
请求真有歧义	问一个简短澄清题 > 猜错重写
规则和任务打架 ("给我列选项")	选项本身就是答案：2-4 个带取舍，推荐置顶
规则和 harness 打架	系统提示词 > 本技能：该报工具调用就报

没有逃生舱的规则集会在边缘场景碎裂：用户问“我的选项有哪些”时如果硬套“给一条路径”，答案直接残废。这和[[Agent指令文件AgentMd]]里的优先级设计、[[质检Gate]]的“硬失败一票否决”是同构思想——**先声明谁让位，再谈规则。**

Q12

我能拿它干什么？

用途	怎么做
① 治自己 AI 助手的废话	装上，会话里 /i-have-adhd；嫌烦 stop adhd mode
② 常开党	摸旗标 <code>~/.claude/.i-have-adhd-always</code> ，每次会话自动注入
③ 抄它的评测架子	你自己调的任何 output-style 提示词，都用 cases+rubric+盲评验证，不许"感觉变好了"
④ fork 调自己的规则	改 SKILL.md → <code>claude plugin marketplace add 你名/i-have-adhd</code>
⑤ 学"格式压力制造幻觉"	给你自己的输出模板加逃生舱条款
⑥ 学多 harness 发布	8 个插头适配同一份核心，适合任何想广撒网的 skill 作者

能力清单：把行为规范写成可安装资产 / Persistence 对抗会话漂移 / 关闭短语给约束出口 / hook 剥头注入 / 旗标文件开关 / 盲评+隔离的 A/B 评测 / blocker 一票否决 / 格式压力幻觉的机制认识。

动手实验（全部真实运行，零 API 费用）

exp1 SKILL.md 结构契约体检 (`exercise/exp1_skill_anatomy.py`)：把 142 行 Markdown 当配置文件逐项机检——frontmatter 四字段、关闭短语与扩展 STOP_PHRASES 逐字对账、10 条规则标题逐一比对、6 条逃生舱、规则 9“不得删分析”红线。**结果 28/28 PASS。**

✅ 预期：✅ i-have-adhd SKILL.md 结构契约体检: 28/28 PASS (关闭短语集合 == 扩展实现)

exp2 去埋葬体检器 (`exp2_debury_linter.py`)：把 Pre-send check 五删检查翻译成 Python 正则，对 README 官方 Before/After + 自造正反例盲测。**结果 4/4**：Before 抓下 3 违规（开场白/结尾客套/by the way），After 放行，自造坏例抓下 4 违规（+对冲词+习语 circle back）。

✅ 预期：[OK] README 官方 Before（应被抓）：抓下 → [OK] After: 放行 → 总体: PASS

exp3 always-on hook 等价复刻 (`exp3_alwayson_replica.py`)：Python 逐字移植 always-on.mjs 的剥头正则，4 组边界夹具（CRLF/无 frontmatter/正文含 --- 不误删/只有 frontmatter）+ 无旗标路径。**结果 10/10 PASS。**（上游 .mjs 因本机安全策略未直接执行，以源码逐行对照为准，如实记录）

exp4 上游 eval 聚合器验证：validate/plan 直跑 PASS（14 例×3 试×2 条件=84 行计划）；用自造 synthetic_responses.jsonl 跑 measure，delta 手工核算一致（输出 token -75.4%、成本 -40%、字符 -88.6%）；负 token 数据被 ValueError 拒收。

exp5 考卷普查：cases.jsonl 14 用例 / 12 类别 / 风险 4低6中4高 / criteria 共 29 条；RESULTS.md 讨论的 4 个焦点用例全部在册。

⚠ 未跑项：上游 pytest 全套被本机 [Code from External] 安全策略拦截，未绕行；RESULTS.md 复现需花 \$3.59 真实调用，未花。